

Comparison of Stretto and SUQL

LLM-Augmented Query Execution over Heterogeneous Data

Project Context

Target Problem

We want to build a system capable of:

- querying heterogeneous data:
 - PDFs
 - relational tables
 - CSV files
 - text collections
 - semi-structured documents
- answering natural language questions
- retrieving correct rows, values, and evidence
- supporting:
 - joins
 - filtering
 - aggregation
 - ranking
 - multi-document reasoning

SUQL = Query Language + Semantic Parsing

Stretto = Execution Engine + Query Optimization

Core distinction

- SUQL focuses on:
 - translating natural language into executable hybrid queries
- Stretto focuses on:
 - efficient execution of LLM operators inside database pipelines

Paper Goal

Build an efficient execution engine for:

- LLM-based operators
- retrieval pipelines
- semantic transformations
- hybrid data workflows

Main Problem

LLM-augmented pipelines are:

- expensive
- slow
- non-deterministic
- difficult to optimize

Stretto: Core Idea

Main Innovation

Treat LLM pipelines like database query plans.

Instead of:

- ad-hoc agent pipelines

Stretto introduces:

- execution plans
- cost-aware optimization
- caching
- operator scheduling

Key Insight

LLM operations should be optimized similarly to relational query operators.

Stretto Pipeline

- ① User query received
- ② System builds execution graph
- ③ Planner estimates:
 - latency
 - token cost
 - retrieval quality
- ④ Retrieval operators executed
- ⑤ Context compressed and routed
- ⑥ LLM operators scheduled efficiently
- ⑦ Cached intermediate states reused
- ⑧ Final grounded answer returned

Stretto Optimizations

- Query planning
 - optimize execution order
- KV-cache reuse
 - avoid recomputing transformer states
- Retrieval-aware execution
 - reduce unnecessary context
- Hybrid retrieval
 - combine sparse + dense retrieval
- Reranking
 - improve evidence quality before LLM
- Bayesian quality estimation
 - estimate answer reliability

Why Stretto Matters

Without optimization

LLM systems become:

- prohibitively expensive
- high-latency
- unstable
- impossible to scale

This is much closer to:

- production infrastructure

than most academic RAG pipelines.

Stretto provides

- database-inspired execution
- reusable operator abstraction
- scalable orchestration
- context management
- execution-level optimization

Stretto Strengths

- Explicit execution planning
- Cost-aware optimization
- Better scalability
- Efficient handling of:
 - long contexts
 - retrieval chains
 - iterative reasoning
- Better suited for:
 - distributed pipelines
 - production deployments
 - large corpora
- Explicit runtime optimization

Stretto Weaknesses

- Does not define a semantic query language
- Less explicit logical representation than SUQL
- Harder interpretability
- Still retrieval-dependent
- Complex engineering system
- Requires sophisticated infrastructure
- Query semantics less transparent

Important distinction

Stretto optimizes execution, but does not solve semantic parsing directly.

SUQL: Objective

Paper Goal

Build a conversational agent that can query:

- structured data
- unstructured text
- both simultaneously

Main Problem

Existing systems typically:

- only support SQL databases
- or only support text retrieval
- or linearize tables into text embeddings

These approaches fail on hybrid questions such as:

“Find an Italian restaurant with romantic atmosphere”

SUQL: Core Idea

Main Innovation

Extend SQL with free-text primitives.

SUQL Operators

- ANSWER(text, question)
- SUMMARY(text)

Example:

```
SELECT *  
FROM restaurants  
WHERE cuisine = 'Italian'  
AND answer(reviews,  
'is this restaurant romantic?')  
= 'Yes';
```

SUQL Pipeline

- 1 User asks natural language question
- 2 LLM semantic parser converts query into SUQL
- 3 SUQL compiler optimizes query
- 4 Structured filters executed with SQL
- 5 Free-text predicates executed using:
 - embeddings
 - retrieval
 - LLM calls
- 6 Retrieved evidence returned to LLM
- 7 Final grounded answer generated

SUQL Compiler Optimizations

Problem

LLM calls are expensive.

Optimizations

- Predicate ordering
 - execute cheap SQL predicates first
- Lazy evaluation
 - stop after LIMIT satisfied

Example:

- user says “coffee”
- schema contains “coffee & tea”
- classifier resolves mismatch

SUQL Strengths

- Explicit and interpretable execution
- Natural integration with SQL
- Supports hybrid structured/unstructured reasoning
- More scalable than full-table transformer models
- Supports:
 - joins
 - nested queries
 - sorting
 - aggregation
 - ranking

Most important contribution

Treats semantic reasoning as executable query operators.

SUQL Weaknesses

- Heavy dependence on semantic parser quality
- LLM hallucinations can produce incorrect filters
- ANSWER() execution still expensive

Important limitation

SUQL mainly defines:

- representation
- parsing
- logical execution

not a full high-performance execution engine.

SUQL vs Stretto

Aspect	SUQL	Stretto
Main focus	Semantic parsing	Execution optimization
Core abstraction	Extended SQL	Execution engine
Main contribution	Hybrid query language	Efficient runtime system
Structured reasoning	Strong	Moderate
Text reasoning	ANSWER/SUMMARY operators	Retrieval pipelines
Interpretability	High	Medium
Scalability	Medium	High
Optimization depth	Limited	Strong

Comparison for Our Task

Our Task

Question answering over:

- PDFs
- tables
- extracted text

SUQL helps with

- semantic query representation
- joins between text and tables
- interpretable hybrid reasoning
- explicit filtering logic

Stretto helps with

- efficient execution
- context optimization
- retrieval orchestration
- scaling to large corpora

Which One Is Better?

For research prototypes

SUQL is often more useful because:

- query semantics are explicit
- easier debugging
- better interpretability

For production systems

Stretto becomes more important because:

- runtime dominates cost
- LLM execution becomes bottleneck
- scaling matters

Extending SUQL

- Remove conversational memory layer
 - stateless execution
 - reproducible querying
- Explicit join-aware reasoning
 - joins across:
 - tables
 - PDFs
 - extracted entities

Extending Stretto

- Scaling to larger corpora
- Better long-context handling
- Adaptive retrieval depth
- Smarter context compression